

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2

Диалоговая система подбора видеоигр
по курсу «Системы искусственного интеллекта»

Модуль 1. Базы знаний и онтологии

Выполнили:

Якименко Владислав, Р3313

Пахомов Игнат, Р3313

Проверил:

Жданов Андрей Дмитриевич

Содержание

1	Цель и требования	1
2	Система поддержки принятия решений	1
2.1	Постановка задачи и формат диалога	1
2.2	Архитектура и алгоритм	1
2.3	Сценарии использования	2
2.4	Ошибки и ограничения	3
3	Тестирование	3
4	Оценка результатов и заключение	4
5	Источники	4

1 Цель и требования

Разработать систему поддержки принятия решений, которая после небольшого диалога формирует логический запрос к базе знаний и выдаёт рекомендации. Используется games.pl лабораторной 1: 12 игр, 51 унарный факт, 15 бинарных фактов и семь правил. База загружается непосредственно из файла, так что приложение не дублирует её содержимое.

2 Система поддержки принятия решений

2.1 Постановка задачи и формат диалога

Система рекомендует игры на основе предпочтений и двух уточнений. Ввод имеет фиксированную форму «Мне нравятся: RPG, инди». Возраст не включён: исходная база не содержит рейтингов, и приложение не может обосновать возрастную фильтрацию. Пример возраста в задании является иллюстрацией возможных слотов, а не единственно допустимым форматом.

Словарь нормализации объединяет русские и английские названия в пять канонических предпочтений: ролевые игры, инди, головоломки, шутеры и пошаговые стратегии. Например, RPG, РПГ и «ролевая игра» дают один атом `role_playing_game`. Словарь полностью приведён в README приложения. Инди является производственным признаком. Для него используется предикат `indie/1`; жанры проверяются отношением `has_genre/2`.

Первый уточняющий вопрос: режим «один», «вместе» или «любой». Второй: нужна ли игра для новичка («да» или «нет»). Пустой и неизвестный ответ приводит к повтору соответствующего вопроса. Рекомендации выводятся только после завершения диалога.

2.2 Архитектура и алгоритм

Python отвечает за диалог, проверку формата и отображение. Модуль `bridge.pl` загружает `games.pl` первой лабораторной и выполняет логический поиск. Входные параметры передаются JSON через стандартный ввод отдельного процесса SWI-Prolog, результаты возвращаются через стандартный вывод. Командная оболочка не используется; пользовательский текст не подставляется в код цели.

```
INPUT preference line
PARSE complete fixed format
NORMALIZE known aliases; REMOVE duplicates
ASK mode; ASK beginner filter
FOR each game in the knowledge base:
    REQUIRE selected mode
    REQUIRE beginner rule if requested
    COLLECT matching preferences
```

```

KEEP game if at least one preference matched
SORT by match count descending, then game identifier
OUTPUT recommendations and matched conditions

```

Предпочтения объединяются по ИЛИ, а уточнения применяются по И. Оценка кандидата g равна числу совпавших различных предпочтений:

$$s(g) = |\{t \in T : \text{matches}(g, t)\}|.$$

Требуется $s(g) > 0$. Одинаковые оценки упорядочиваются по идентификатору, что делает результат воспроизводимым. В текущих данных комбинации жанров и инди не пересекаются, поэтому фактические оценки в сценариях равны единице. Счёт поддерживает расширение базы, но не изображает вероятности или качество игр.

```

candidate(Tags, Mode, Experience, G, Score, Matched) :-
    game(G), mode_fits(G, Mode),
    experience_fits(G, Experience),
    findall(T, (member(T, Tags), matches(G, T)), Raw),
    sort(Raw, Matched), length(Matched, Score), Score > 0.

```

При N играх и K предпочтениях логическая фильтрация выполняет порядка NK проверок совпадения; сортировка R результатов требует порядка $R \log R$ сравнений. Для 12 игр стоимость запуска интерпретатора важнее самого поиска. Индексацию простых фактов обеспечивает SWI-Prolog; отсечение не требуется.

2.3 Сценарии использования

Каждая строка ниже соответствует реальному вызову моста Prolog, записанному в evidence.json. Начало входной строки во всех случаях — «Мне нравятся:».

Таблица 1: Входы, рекомендации и объяснения

Предпочтения	Уточнения	Результат и основание
RPG	один; новичок	cyberpunk_2077, the_witcher_3: жанр RPG, single_player и recommended_for_beginner.
Инди	вместе; новичок	stardew_valley: indie, multiplayer и правило новичка.
Головоломка	вместе; любой опыт	portal_2: has_genre(puzzle) и multiplayer.
Шутер	один; любой опыт	doom_eternal: has_genre(first_person_shooter) и single_player.
Стратегия	любой режим; любой опыт	civilization_vi: has_genre(turn_based_strategy).

Предпочтения	Уточнения	Результат и основание
RPG	вместе; любой опыт	Пустой список: ни у одной игры с записанным RPG нет multiplayer.
Шутер	один; новичок	Пустой список: Doom Eternal отмечена difficult и исключена правилом новичка.
RPG, инди	один; новичок	cyberpunk_2077, stardew_valley, the_witcher_3: каждый совпал с одним предпочтением.

Например, второй сценарий формирует запрос:

```
candidate([indie],multi,beginner,G,S,M)
```

Для Stardew Valley проверяются признаки indie и multiplayer, затем правило recommended_for_beginner. Итоговый балл равен 1, а список объясняющих совпадений равен [indie].

2.4 Ошибки и ограничения

Неизвестное предпочтение «гонки», пустая строка и завершающая запятая отвергаются до запуска Prolog. Повторение RPG и РПГ не увеличивает балл. При конфликте режима и признаков система сообщает, что совпадений в БЗ нет, и предлагает изменить режим или снять фильтр новичка. Отсутствие swipl приводит к инструкции установить интерпретатор; на выполнение запроса установлен таймаут 15 секунд. EOF и Ctrl+C завершают диалог.

Пустой список не доказывает отсутствие подходящей игры в мире. В частности, жанр Baldur's Gate 3 в этой БЗ не задан, поэтому система не рекомендует её по RPG. Такой результат согласован с данными и явно документирован. Признак «подходит новичку» не является рекомендацией по возрасту.

3 Тестирование

Семь методов Python unittest успешно проверяют нормализацию, восемь интеграционных сценариев, шесть ошибочных входных строк, четыре неверных набора параметров API, отсутствие swipl, восстановление диалога после ошибок и EOF. Интеграционные тесты запускают реальный SWI-Prolog 10.0.2. Результаты всех восьми сценариев записаны в evidence.json.

```
python3 lab_2/recommender.py
python3 -m unittest discover -s lab_2 -v
```

Для запуска нужны Python 3.9+ и SWI-Prolog в PATH. Python-зависимости для DSS ограничены стандартной библиотекой.

4 Оценка результатов и заключение

Программа выполняет полный цикл: разбирает фиксированный ввод, уточняет режим и опыт, выполняет логические запросы и объясняет результат. Пустая выборка не трактуется как сбой; пользователь получает предложение ослабить условия. Неверный ввод не передаётся в интерпретатор.

Главное ограничение — неполнота жанровых фактов, а не скорость алгоритма. На текущем наборе все совпадения имеют балл 1, поэтому дополнительный порядок задаётся именем игры. Для расширения системы нужны новые подтверждённые факты, а не искусственное увеличение оценок. Возраст и оборудование не анализируются, поскольку соответствующих сведений в базе нет.

5 Источники

Методическое пособие «Системы искусственного интеллекта», ИТМО, 2024, с. 57–58, приложение 2: <https://books.ifmo.ru/file/pdf/3308.pdf>. Задание «Системы ИИ.docx», лабораторная 2 и шаблон отчёта по модулю 1.